

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

1. Erlang и Elixir



ERLANG

1.1. Erlang

Erlang е функционален език за програмиране, създаден за изграждане на надеждни, конкурентни и разпределени системи. Разработен е първоначално за телекомуникационни приложения, където се изисква висока надеждност и непрекъсната работа. Езикът поддържа едновременност чрез леки процеси и обмен на съобщения. Основен принцип е „let it crash“, което улеснява изграждането на устойчиви системи. Erlang използва неизменяеми данни и функционален стил.



elixir

1.2. Elixir

Elixir е съвременен функционален език, изграден върху виртуалната машина BEAM. Той комбинира мощта на Erlang с по-модерен и четим синтаксис. Elixir е подходящ за уеб приложения, микросървиси и разпределени системи. Поддържа функционално и частично метапрограмиране чрез макроси. Използва екосистемата и библиотеките на Erlang.

1.2.1. Elixir функции

Функциите са основна абстракция в Elixir. Те са първокласни стойности и могат да се предават като аргументи. Поддържат се наименувани и анонимни функции. Elixir използва pattern matching при дефиниране на функции. Функциите са по подразбиране чисти и без странични ефекти.

1.2.1.1. Модули

Синтаксис на модул

```
defmodule ModuleName do
  def function_name(arg1, arg2) do
    # тяло на функцията
  end
end
```

Синтаксис на модули и йерархия в Elixir:

- Модул се дефинира с ключовата дума defmodule
- Йерархията се задава чрез точка (.) в името на модула
- Всеки следващ елемент представлява „подмодул“ на предходния
- Името на файла обикновено следва последната част от името на модула

Синтаксис на йерархия от модули

```
defmodule ParentModule.SubModule do
  def function_name(arg1, arg2) do
    # тяло на функцията
  end
end
```

Синтаксис на дълбока йерархия

```
defmodule App.Domain.Context.ModuleName do
  def function_name(arg1, arg2) do
    # тяло на функцията
  end
end
```

Следователно: Модулната йерархия е като папки в директория
MyApp/ Web/ Controllers/ UserController.

Общи правила за наименоване на модули в Elixir:

- Имената на модулите се пишат в PascalCase
- Допустими са латински букви (A–Z, a–z)
- Допустими са цифри (0–9), но не като първи символ
- За йерархия между модули се използва точка (.)
- Всяка част от името започва с главна буква
- Не се допускат интервали и специални символи (_, -, @, ! и др.)
- Името на модула трябва логически да съответства на името на файла

1.2.1.1. Функции

Синтаксис на наименовани функции (Elixir)

Наименованите функции се дефинират в модул и имат име и арност (брой аргументи).

```
defmodule ModuleName do
  def function_name(arg1, arg2) do
    # тяло на функцията
  end
end
```

Кратък едноредов синтаксис:

```
def function_name(arg1, arg2), do: expression
```

Правила за именуване на наименовани функции (Elixir)

- Името на функцията започва с малка буква.
- Допускат се малки букви, цифри и долна черта (_).
- Препоръчва се използване на snake_case за по-добра четимост.
- Името не може да съвпада със запазена ключова дума.
- Функциите с логически резултат често завършват с ?.
- Функциите, които могат да предизвикат грешка, често завършват с !.
- Името трябва да бъде описателно и да отразява предназначението на функцията.
- Две функции могат да имат едно и също име, ако имат различна арност.
- Името на функцията се дефинира винаги вътре в модул.
- Препоръчва се избягване на твърде кратки или неясни имена.

Според видимостта те биват public (def) и private (defp)

```
defmodule Math do
  # Публична функция
  def add(a, b) do
    a + b
  end

  # Частна функция
  defp multiply(a, b) do
    a * b
  end
end
```

Според броя на аргументите (арност, arity)

```
defmodule Example do
  def greet(), do: "Hello!"
  def greet(name), do: "Hello, #{name}!"
end

Example.greet()    # => "Hello!"
Example.greet("Ani") # => "Hello, Ani!"
```

Кратки функции

```
def square(x), do: x * x
```

Според начина на извикване

А. Могат да бъдат извикани отвън модула.

```
defmodule Math do
  def add(a, b), do: a + b
end
```

```
Math.add(2, 3) # => 5
```

Б. Могат да се използват само вътре в модула. Не се виждат и не могат да се извикат извън модула.

```
defmodule Math do
  defp multiply(a, b), do: a * b
  def square_sum(a, b), do: multiply(a, a) + multiply(b, b)
end
```

```
Math.multiply(2, 3) # => грешка!
```

```
Math.square_sum(2, 3) # => 13
```

Синтаксис на анонимни функции (Elixir)

Анонимните функции нямат име и обикновено се присвояват на променлива или се подават като аргумент.

Синтаксис

```
fn arg1, arg2 ->
  expression
end
```

Извикване на анонимна функция

```
fun = fn a, b -> a + b end
fun.(2, 3)
```

Съкратен синтаксис (capture оператор)

```
add = &(&1 + &2)
add.(2, 3)
```

В Elixir концепцията за „видимост“ важи само за **именованите функции (def и defp)** в рамките на модул. **Анонимните функции (fn ... end)** нямат собствено име в модула и нямат традиционна публична или частна **видимост** — тяхната достъпност зависи от това къде е съхранена или предадена променливата, която ги държи.

Ако анонимната функция е присвоена на локална променлива в рамките на функция:

```
defmodule Demo do
  def run do
    hidden = fn -> IO.puts("I am hidden") end
    hidden.()
  end
end
```

Тази функция е „скрита“ за външния свят — извън `run/0` няма начин да се вижда. Това е еквивалент на `private score` (т.е. частна област на видимост, която се отнася до контекста или блок от код, където определени променливи, функции или ресурси **са достъпни само вътре в този блок и не могат да бъдат достъпвани отвън**), но формално не е `defp`, а просто локална променлива.

Ако анонимната функция се присвои на **публична променлива на модул**, може да се използва отвън:

```
defmodule Demo do
  @public_fn fn -> IO.puts("I am public") end
  def call_public, do: @public_fn.()
end
Demo.call_public() # => "I am public"
```

Тук достъпът е „публичен“ само през другите публични функции (`call_public/0`), но самата анонимна функция **не е може директно да се извика от вън** като `Demo.@public_fn.()` — Elixir не позволява директен достъп до module attributes като функции.

Анонимната функция сама по себе си не може да бъде истински публична или скрита в смисъла на `def/defp`. Може да се направи „скрита“ чрез локална променлива, или да се „сподели“ чрез публична функция. Формата на достъп зависи от **scope-a**, в който е дефинирана, а не от ключова дума.

В кода е използван символа `@`. В Elixir символът се използва за module attributes (атрибути на модул). Те са като метаданни или константи, които се свързват с конкретен модул и имат различни приложения.

Можем да присвоим анонимна функция на `@` атрибут, за да се използва в целия модул:

```
defmodule Demo do
  @double fn x -> x * 2 end

  def double(x), do: @double.(x)
end

Demo.double(5) #=> 10
```

Без `@` функцията би била просто локална променлива и достъпна само в блока, където е дефинирана.

Предназначение на @

- За константи и споделени стойности, които трябва да се ползват в целия модул.
- За метаданни и конфигурации, които инструменти, библиотеки или компилаторът могат да използват.
- За да направим локални променливи достъпни на ниво модул, без да са публични функции.

Таблица 1. Видимост-дефиниране-достъп (обобщение)

Видимост	Дефиниране	Достъп отвън
Публична	<code>def</code>	Да
Скрита	<code>defp</code>	Не
Анонимна	<code>fn ... end</code>	Достъпна през променлива

Според типа на връщана стойност при използване на функции

В Elixir няма строго типизирани функции като в C# или Java – **езикът е динамично типизиран**. Това означава, че функциите могат да връщат всякакви стойности, а типът се определя по време на изпълнение. Въпреки това можем да говорим за видове връщани стойности и как те се използват.

1) Основни типове на връщана стойност

а) Атоми

Атомите са символни константи, започващи с `:`. Често се използват за маркиране на статуси или резултати. Когато казваме, че в Elixir атомите „често се използват за маркиране на статуси“, имаме предвид, че те служат като символични индикатори за състоянието или резултата на операция. Тоест, вместо да връщаме произволен низ или число, използваме атом, за да кажем на друг код „това е успех“, „това е грешка“ или „текущото състояние е такова“.

Например:

```
def ok_example do
  :ok
end

ok_example() #=> :ok
```

```
def divide(a, b) do
  if b == 0 do
    { :error, :division_by_zero } # атом :error маркира проблем
  else
    { :ok, a / b } # атом :ok маркира успешен резултат
  end
end
```

`:ok` и `:error` са атомите, които показват статуса на операцията.

Предимството на атомите е, че са по-ефективни и безопасни, отколкото стрингове като `"ok"` или `"error"`.

```
case divide(10, 2) do
```

```
{:ok, result} -> IO.puts("Резултат: #{result}")
{:error, reason} -> IO.puts("Грешка: #{reason}")
end
```

Пример при използване на файл

```
File.read("file.txt")
#=> {:ok, "съдържание"} или {:error, :enoent}
```

Генерално в Elixir много функции връщат `{:ok, value}` или `{:error, reason}`. Така кодът лесно разграничава успешни и неуспешни операции, без да проверява числови кодове или текстови низове.

Таблица 2. Най-често използвани атоми

Атом	Значение / Статус	Пример
<code>:ok</code>	Операцията е успешна	<code>{:ok, result} File.read("file.txt") #=> {:ok, "data"}</code>
<code>:error</code>	Възникнала е грешка	<code>{:error, :enoent} divide(10, 0) #=> {:error, :division_by_zero}</code>
<code>:ok?</code>	Булев успех (понякога)	В собствени функции за кратко индикиране на успех/неуспех
<code>:not_found</code>	Ресурсът не е намерен	<code>get_user(id) #=> {:error, :not_found}</code>
<code>:invalid</code>	Невалидни данни или операция	<code>validate_input(data) #=> {:error, :invalid}</code>
<code>:timeout</code>	Операцията е изтекла	<code>request_api() #=> {:error, :timeout}</code>
<code>:unauthorized</code>	Липса на права	<code>access_page(user) #=> {:error, :unauthorized}</code>
<code>:ok_or_skip</code>	Успех или пропускане (по избор)	Използва се при специфични потоци, напр. в Enum
<code>:done</code>	Завършено успешно	Често в процеси/генериране на задачи

ПРЕДИМСТВА от използване на атоми

- **Яснота** – не се губи смисълът на резултата като при числа или стрингове.
- **Скорост** – атомите се сравняват много бързо.
- **Безопасност** – по-трудно е да се направи грешка при проверка на статуси.
- **Конвенция в Elixir** – почти всички стандартни библиотеки връщат `{:ok, value}` или `{:error, reason}`.

б) Числа

Цели числа (integer) и десетични (float) могат да се връщат директно:

```
def add(a, b), do: a + b
add(2, 3) #=> 5
```

с) Стрингове

Връщат текст:

```
def greet(name), do: "Hello, #{name}!"
greet("Ani") #=> "Hello, Ani!"
```

д) Булеви стойности

true или false:

```
def is_even(n), do: rem(n, 2) == 0
is_even(4) #=> true
```

е) Списъци и кортежи

Списъци: `[1, 2, 3]`

Кортежи: `{status, value}`

```
def result_example do
  {:ok, 42}
end
result_example() #=> {:ok, 42}
```

Кортежите често се използват за tuple-based статус: `{:ok, value}` или `{:error, reason}`.

f) Map и Struct

За по-сложни данни:

```
def user_info do
  %{name: "Ani", age: 25}
end
user_info() # => %{name: "Ani", age: 25}
```

g) Функции

В Elixir може да върнеш дори друга функция (first-class функции):

```
def make_multiplier(n) do
  fn x -> x * n end
end
double = make_multiplier(2)
double.(5) # => 10
```

2) Важни особености

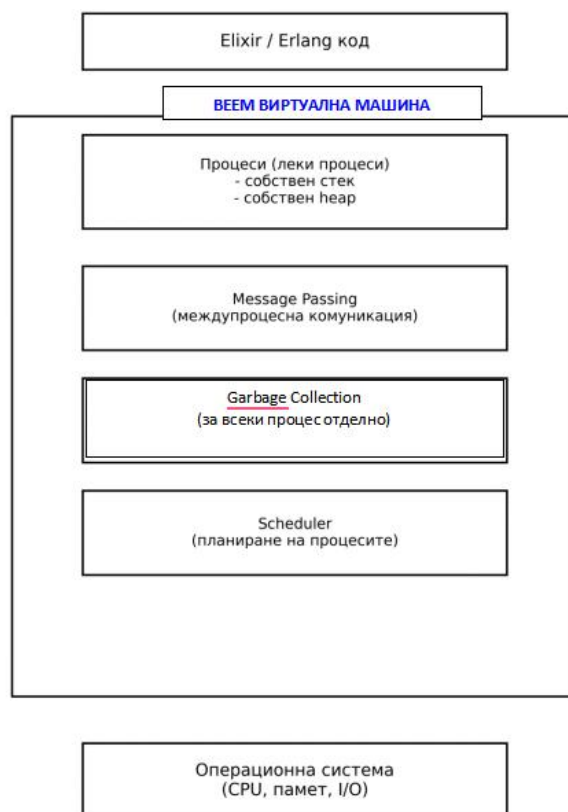
Функциите винаги връщат стойност – дори **IO.puts** връща **:ok**. Последната изпълнена стойност в тялото на функцията е връщаната стойност:

```
def example do
  x = 10
  x * 2 # това е връщаната стойност, не x
end
example() # => 20
```

Типът не се декларира – компилаторът не проверява. Типът се използва динамично по време на изпълнение.

2. Характеристика и особености на BEAM виртуалната машина

BEAM е виртуалната машина, върху която се изпълняват Erlang и Elixir. Тя е оптимизирана за конкурентност и разпределени изчисления. BEAM използва леки процеси вместо нишки на операционната система. Всеки процес има собствена памет и не споделя състояние с други процеси. Комуникацията се осъществява чрез изпращане на съобщения. Виртуалната машина осигурява висока устойчивост и възстановяване при грешки.



ФИГ.1

- Elixir / Erlang код

Това е изходният код, написан на Elixir или Erlang. Той се компилира до байткод, който се изпълнява от виртуалната машина BEAM.

- BEAM виртуална машина

BEAM е средата за изпълнение на Elixir и Erlang програми. Тя управлява процесите, паметта, планирането и комуникацията между тях.

● Процеси (леки процеси)

BEAM използва леки процеси, които са много “по-евтини” от нишките на операционната система. Всеки процес има собствен стек и heap и не споделя памет с други процеси.

Това означава, че процесите в BEAM не са същите като нишките (threads) на операционната система, а са вътрешни, много по-леки за изпълнение, управлявани изцяло от BEAM.

Например:

- Лек процес в BEAM заема много малко памет (няколко килобайта) и се създава изключително бързо.
- Нишка на операционната система е „тежка“ – създава се по-бавно и използва повече ресурси.

Затова BEAM може да изпълнява стотици хиляди или милиони процеси едновременно, което би било невъзможно с OS нишки.

Пример:

В Elixir всеки `spawn` създава нов BEAM процес, не нова нишка на ОС.

● Междупроцесна комуникация (Message Passing)

Процесите комуникират чрез изпращане и получаване на съобщения. Този модел елиминира необходимостта от споделено състояние и заключвания.

● Garbage collection

BEAM използва автоматично управление на паметта чрез garbage collection. Всеки процес има собствен garbage collection, което намалява паузите и повишава ефективността.

Докато в програмния език C++ съществува понятието "деструктор" - метод който "почиства" преди даден обект да бъде унищожен. Този метод е често използван там, като предоставя възможност на програмистът сам контролира кога един обект да бъде изтрит (чрез "обратния" на new оператор delete).

- **Scheduler (Планиращ механизъм)**

Scheduler-ът разпределя процесорното време между активните процеси. Той осигурява справедливо и конкурентно изпълнение.

- Операционна система

Операционната система предоставя хардуерните ресурси като процесор, памет и вход/изход. ВЕАМ работи върху нея като абстракция и не зависи от конкретната платформа.

3. Разпределение на паметта и начин на извод

3.1. Въведение

ВЕАМ използва специфичен модел за управление на паметта. Той е съобразен с функционалния стил и конкурентното изпълнение. Всеки процес има собствен стек и heap.

3.2. ВЕАМ виртуална машина

3.2.1. Области на данни

Паметта е разделена на няколко логически области. Те включват стек, heap и области за съобщения. Това изолира процесите и предотвратява състезателни условия.

3.2.2. Обекти на данни

Данните са неизменяеми. Обектите се копират при предаване между процеси. Това осигурява безопасност и предсказуемост.

3.2.3. Регистри

ВЕАМ използва виртуални регистри. Те се използват за временни стойности и аргументи на функции.

3.2.4. Извиквания на функции

Извикванията са оптимизирани чрез tail-call optimization. Това позволява ефективна рекурсия без нарастване на стека.

3.2.5. Едновременност

Едновременността се реализира чрез процеси. Процесите са евтини и могат да бъдат хиляди или милиони. Комуникацията е асинхронна.

Можем да обобщим т.3.2.1. до 3.2.5.

Паметта и данните са изолирани и безопасни, като обектите се копират при предаване, подобно на пощенските пратки между отделни офиси. Процесите са леки и комуникират асинхронно, подобно на екипите в компания, които работят независимо, но си изпращат мейли.

3.3. Инструкции на виртуалната машина ВЕАМ

3.3.1. Формат на инструкции - Инструкциите са бинарни и компактни. Те са оптимизирани за бързо изпълнение.

3.3.1.1. Инструкции за форматиране - Използват се за управление на подреждането на данни и стек.

3.3.1.2. Процедурни инструкции - Отговарят за извикване и завършване на функции.

3.3.1.3. Инструкции за поставяне (put) - Използват се при създаване на структури от данни.

3.3.1.4. Инструкции за получаване (get) - Служат за извличане на стойности от структури.

3.3.1.5. Test инструкции - Използват се за проверки и pattern matching.

3.3.1.6. Инструкции за преместване (move) - Копират стойности между регистри и памет.

3.3.1.7. Обработка на изключения - Позволяват контролирано управление на грешки.

3.3.1.8. Междупроцесна комуникация - Реализира се чрез изпращане и получаване на съобщения.

3.3.1.9. Примитиви - Нискониво инструкции за базови операции.

3.3.1.10. Вградени функции - Функции, реализирани директно във виртуалната машина.

3.3.1.11. Аритметични изрази - Поддържат стандартни аритметични операции.

3.3.1.12. Сравнения на срокове - Използват се при работа с време и таймери.

3.3.1.13. Инструкции за индексване - Осигуряват достъп до елементи в структури.

3.3.1.14. Сгъване на инструкции - Оптимизация чрез комбиниране на инструкции.

Разлика между компилатор и интерпретатор

- Компилаторът превежда целия изходен код наведнъж в машинен код (изпълним файл, напр. .exe), което води до по-бърза работа, но изисква време за компилация.
- Интерпретаторът чете и изпълнява кода ред по ред в реално време, което позволява по-бързи промени (скриптове), но програмите работят по-бавно.

Примери

- C++ е компилиран език – кодът се компилира директно в машинен код за конкретната платформа.
- Elixir е компилиран език – кодът му се компилира до байткод за виртуалната машина BEAM.

Но Elixir има 2 типа разширения `ex` и `exs`. Тогава какъв е той?

При Elixir в скриптов режим (`elixir file.exs`) кодът **СЪЩО** се компилира, НО временно в паметта до байткод за виртуална машина BEAM, **КАТО НЕ СЕ създава отделен .beam файл**, видим за потребителя. **Това е т.нар. implicit / just-in-time компилация.**

т.е. кода при

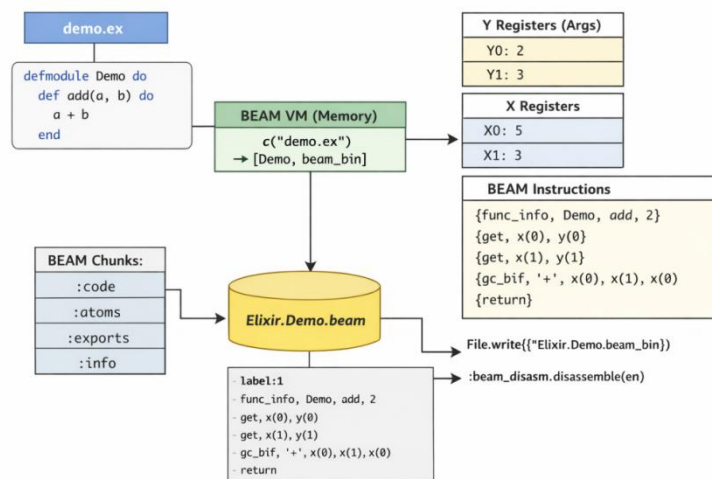
- C++ - се компилира предварително до машинен код, който после се стартира директно.
- Elixir - се компилира до байткод за виртуална машина (BEAM), дори при скриптове.

1) Създайте модул `add`

```
defmodule Demo do
  def add(a, b) do
    a + b
  end
end
```

Целта на функцията е да събере две числа.

1. Съхранете файла под име `demo.ex`, той реално съдържа модул с функция `add/2`. Този файл е изходният код на Elixir, който ще **компилираме**.
2. Чрез командата `Code.compile_file/1` компилирайте кода в **BEAM байт код**, по този начин модул `Demo` ще се зареди в паметта на виртуалната машина.
3. BEAM байт кодът представлява инструкции, които VM изпълнява директно. Ако искаме файлът `Elixir.Demo.beam` да се съхрани на диска, може да го запишем чрез командата `File.write!("Elixir.Demo.beam", beam_bin)`.
4. BEAM файлът съдържа различни `chunks` (чънкове) — логически сегменти с данни, като `:code` за инструкции, `:atoms` за атомите, `:exports` за функциите и др. Чънковете позволяват на VM или инструментите да четат отделни части от файла без да зарежда целия модул. С `:beam_lib.chunks/2` можем да вземем конкретен чънк, например `:code`, за дисасемблиране. Като използваме командата `:beam_disasm.file('Elixir.Demo.beam')` (т.е. `:beam_disasm.disassemble/1`) BEAM кода се превръща в четими инструкции като `get`, `gc_bif '+'` и `return`. Така се вижда как VM обработва аргументите, извършва изчисления и връща резултата, а чънковете дават структурата на самия BEAM файл.



Фиг.2

Схемата от Фиг.2. показва как Elixir файл (demo.ex) се компилира от BEAM VM до .beam байткод, който съдържа инструкции, регистри и метаданни. Функцията add/2 се превежда в BEAM инструкции, използващи X и Y регистри за аргументи и временни стойности. Полученият .beam файл може да се съхрани, зареди и дори дизасемблира за анализ.

Команди в Command Prompt:

1. Преминуваме в интерактивен режим на работа

```
iex
```

2. Записваме инструкцията (интерактивен режим)

```
[{Demo, beam_bin}] = Code.compile_file("demo.ex")
```

3. Записваме инструкцията (интерактивен режим)

```
File.write!("Elixir.Demo.beam", beam_bin)
```

Резултатът на екрана е следния:

```
iex(1)> [{Demo, beam_bin}] = Code.compile_file("demo.ex")
[
  {Demo,
    <<70, 79, 82, 49, 0, 0, 6, 32, 66, 69, 65, 77, 65, 116, 85, 56, 0, 0, 0, 169,
      255, 255, 255, 238, 176, 69, 108, 105, 120, 105, 114, 46, 68, 101, 109,
      111, 128, 95, 95, 105, 110, 102, 111, 95, 95, 160, 97, 116, 116, 114, 105,
      98, 117, 116, 101, 115, 112, 99, 111, 109, 112, 105, 108, 101, 160, 100,
      101, 112, 114, 101, 99, 97, 116, 101, 100, 176, 101, 120, 112, 111, 114,
      116, 115, 95, 109, 100, 53, 144, 102, 117, 110, 99, 116, 105, 111, 110,
      115, ...>>}]
]
```

4. Записваме инструкцията (излизаме от интерактивен режим)

```
dir Elixir.Demo.beam
```

```
C:\Users\vboji\Desktop\Elixir>dir Elixir.Demo.beam
Volume in drive C is Windows
Volume Serial Number is C8FB-1414

Directory of C:\Users\vboji\Desktop\Elixir

25.02.2026 г.   00:24                1 576 Elixir.Demo.beam
               1 File(s)                1 576 bytes
               0 Dir(s)  24 232 288 256 bytes free
```

Анализ

Компилираме модула и вземаме BEAM кода

```
[{Demo, beam_bin}] = Code.compile_file("demo.ex")
```

Code.compile_file("demo.ex") компилира Elixir файла в BEAM байт код, зарежда модул Demo в паметта на VM.

Връща списък с tuple-и: [{Demo, beam_bin}], където:

- Demo е името на модула
- beam_bin е машинния код (BEAM байт код), който виртуалната машина изпълнява.

Т.е. „Компилиране на кода и вземаме изпълнимия байт код“.

Записваме BEAM код във файл

```
File.write!("Elixir.Demo.beam", beam_bin)
```

Вземаме `beam_bin` (байт код от паметта) и го записваме като файл на диска `Elixir.Demo.beam`. След това файлът вече може да се дисасемблира или да се използва от други Elixir/Erlang програми. Т.е. „Вземаме компилирания код от паметта и го съхраняваме като файл“.

Проверка дали файлът е създаден става чрез инструкцията:

```
dir Elixir.Demo.beam
```

Това е CMD/ PowerShell команда, която показва дали файлът съществува и неговия размер. Ако файлът е там - всичко е успешно компилирано и записано. Т.е. „Потвърждение, че файлът с BEAM код е наличен“.

Какво е асемблиране и какво е дисасемблиране?

- **Асемблиране** е процесът, при който Elixir код се компилира в BEAM байткод, готов за изпълнение от виртуалната машина.
- **Дисасемблиране** е обратният процес — байткодът се превръща обратно в четим списък от инструкции, за да може човек да види как VM изпълнява кода.

Дисасемблиране става чрез инструкцията:

```
result = :beam_disasm.file('Elixir.Demo.beam')
```

```

C:\Users\vboji\Desktop\Elixir>iex
Erlang/OTP 28 [erts-16.2] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit:ns]

Interactive Elixir (1.19.5) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> result = :beam_disasm.file('Elixir.Demo.beam')
warning: using single-quoted strings to represent charlists is deprecated.
Use ~c"" if you indeed want a charlist or use "" instead.
You may run "mix format --migrate" to change all single-quoted
strings to use the ~c sigil and fix this warning.
└─ iex:1:28
iex(1)> result = :beam_disasm.file('Elixir.Demo.beam')
{:beam_file, Demo,
 [
  {:__info__, 1, 2},
  {add, 2, 11},
  {module_info, 0, 13},
  {module_info, 1, 15},
 ], [vsr: [228854796673025461814249890148621476178]]],
 [
  version: ~c"9.0.4",
  options: [:no_spawn_compiler_process, :from_core, :no_core_prepare,
    :no_auto_import],
  source: ~c"c:\Users\vboji\Desktop\Elixir\demo.ex"
 ],
 [
  {:function, :__info__, 1, 2,
   [
    {:label, 1},
    {:line, 0},
    {:func_info, {:atom, Demo}, {:atom, :__info__}, 1},
    {:label, 2},
    {:select_val, {:x, 0}, {:f, 9}},
    {:list,
     [
      atom: :attributes,
      f: 8,
      atom: :compile,
      f: 8,
      atom: :deprecated,
      f: 7,
      atom: :exports_md5,
      f: 6,
      atom: :functions,
      f: 5,
      atom: :macros,
      f: 7,
      atom: :md5,
      f: 8,
      atom: :module,
      f: 4,
      atom: :struct,
      f: 3
     ]},
    {:label, 3},
    {:move, {:atom, nil}, {:x, 0}},
    :return,
    {:label, 4},
    {...},
    ...
   ]},
  ...
 ]}

```

Дисасемблирането на Demo модула в 5 стъпки

1. `:beam_disasm.file/1` чете .beam файла и връща структура `{:beam_file, Module, exports, attributes, compile_opts, functions}`, която съдържа всички данни за компилирания модул.
2. Списъкът `exports` показва какви функции са достъпни извън модула, например `:add/2` или `:module_info/0`.
3. `attributes` съдържа допълнителна информация за компилация, версия, изходен файл и други метаданни.
4. Полето `functions` съдържа байткода на всяка функция, под формата на инструкции като `:label`, `:move`, `:select_val` и `:return`.
5. Когато файла се „дисасемблира“, се вижда как Elixir е превърнал Elixir функциите в BEAM инструкции, които виртуалната машина изпълнява.

Извод

Erlang и Elixir използват BEAM като мощна платформа за конкурентно и разпределено програмиране. BEAM осигурява ефективно управление на паметта, процеси и съобщения. Тази архитектура прави системите устойчиви, мащабируеми и надеждни.